

AI-Powered Smart Grocery & Nutrition Planner

Presentation by:

Amruta Kirtikumar Khot – ak3792@rit.edu

Jhanavi Lingamneni – jl7903@rit.edu

Jiayu Yi – jy1135@rit.edu

R·I·T

ROCHESTER INSTITUTE OF TECHNOLOGY

Problem Statement

Challenges in Meal Planning

- Difficulty deciding meals using available ingredients
- Lack of visibility into pantry contents
- Reliance on last-minute or unhealthy food choices

Limitations of Existing Systems

- Do not consider real-time pantry ingredients
- Ignore dietary preferences and nutrition
- No personalization based on user behaviour

Impact

- Food waste due to unused ingredients
- Duplicate grocery purchases
- Increased expenses and inefficient planning.



Proposed System

- Input: user pantry ingredients
- Output:

best matching recipes

missing ingredients

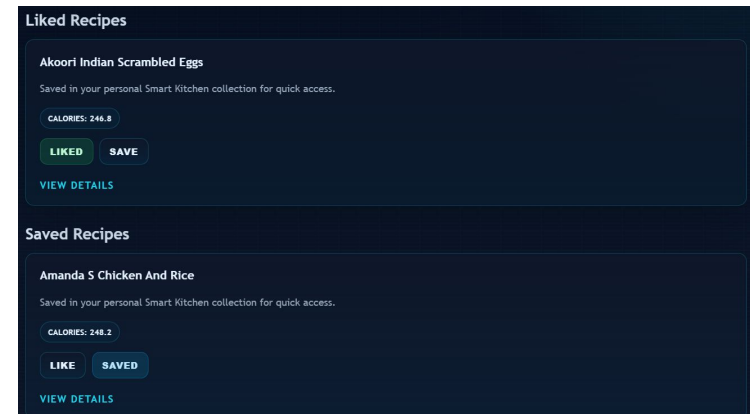
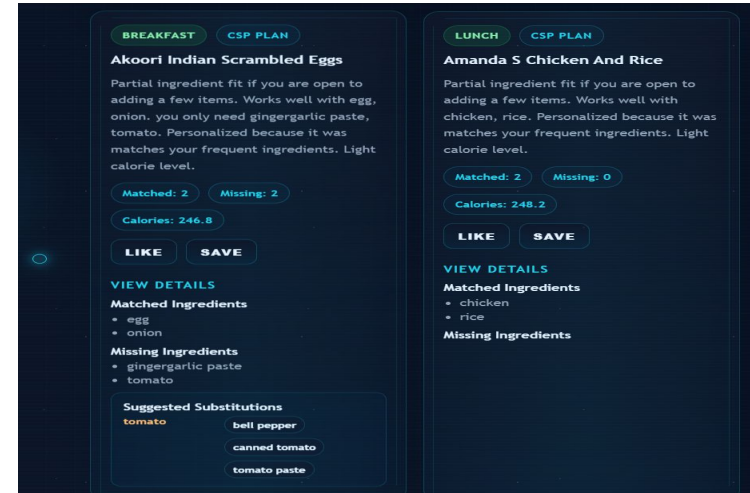
nutrition insights

- Additional features:

dietary filters

personalization

AI chat assistant



Dataset



Dataset Source

Food.com Recipes Dataset

Community-contributed recipes with ingredients, nutrition, and instructions.



Files Used



RAW_recipes.csv

Original raw dataset
(~231K recipes)



recipes_clean.json

Preprocessed and cleaned dataset
(~231K recipes)

Key Fields Overview

Field	Description
id	Unique recipe identifier
name	Recipe title
ingredients	List of ingredient strings
nutrition	Nutritional information (calories, fat, protein, etc.)
steps	List of cooking instructions
n_ingredients	Count of ingredients in the recipe



Why this dataset fits our project

- Rich ingredient lists enable pantry matching.
- Nutrition information supports nutrition-aware ranking.
- Step-by-step instructions allow recipe explanation.
- Recipe complexity (via n_ingredients) helps filter unreasonable recipes for real-world usability.



What each field supports

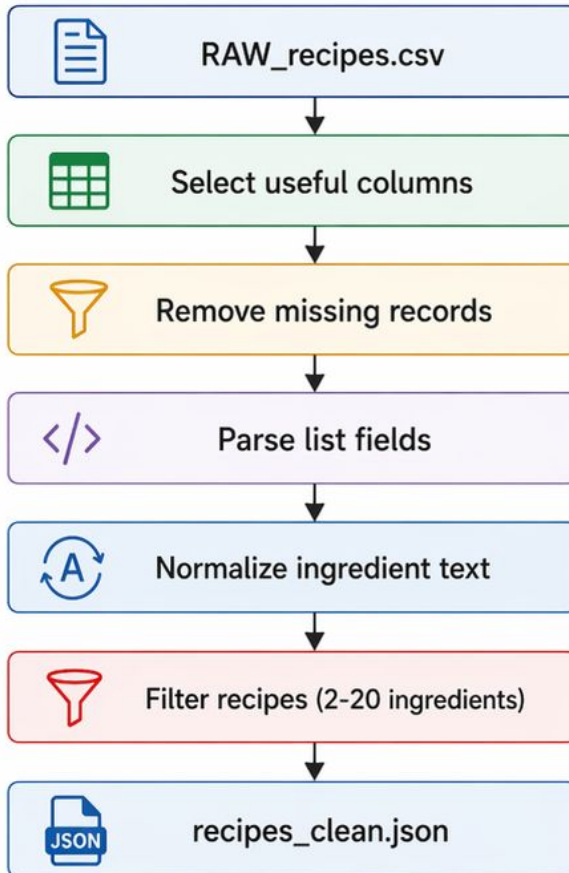
- **id**: Stable reference for linking and evaluation.
- **name**: Used for display and user-facing results.
- **ingredients**: Core for matching against user pantry and computing similarity.
- **nutrition**: Enables health-aware recommendations (e.g., low-calorie, high-protein).
- **steps**: Provides interpretability and supports explanation of recommendations.
- **n_ingredients**: Proxy for recipe complexity and used in filtering and features.



Prototype scope

- **Prototype subset**: 5,000 recipes sampled from the cleaned dataset.
- Sufficient diversity for training and evaluation.
- Ensures faster iteration and reproducibility.

Preprocessing Pipeline



Why preprocessing matters

- Raw data is noisy, inconsistent, and incomplete.
- Cleaning improves data quality and consistency.
- Better data leads to better model performance and generalization.



Key actions

- Lowercase all ingredient text.
- Remove special characters and extra spaces.
- Parse ingredient, nutrition, and steps lists.
- Normalize units and standardize text formats.
- Filter recipes to keep 2–20 ingredients.
- Improve consistency and reduce noise.



Output

- Cleaned dataset saved as recipes_clean.json.
- Structured, consistent, and model-ready data.
- Ready for feature engineering and ML models.

Algorithms Used

- Content-based recommendation
- Constraint Satisfaction Problem
- Heuristic Search
- Logistic regression
- MLP
- Decision Tree
- LLM-based assistant

Constraint Satisfaction Approach

- ❖ Why CSP?
- ❖ How it works
 1. Define variables
 2. Assign recipes using backtracking
 3. At each step check constraints
 4. If constraint fails -> backtrack (undo choice)
 5. Continue until a complete valid assignment is found
- ❖ CSP provides accuracy and completeness but sacrifices speed and scalability

Screenshots of CSP Approach

Smart AI Meal Planner (CSP)

Max Calories



Min Protein (g)



Select Diet

vegetarian



Generate Meal Plan 

Meal Plan

Breakfast: grilled ranch bread

Lunch: arriba baked winter squash mexican style

Dinner: a bit different breakfast pizza



Nutrition

Calories: 1874.7

Protein: 104.0



Plan Score

100



Grocery List

```
▼ {  
  "butter" : 2  
  "dry ranch dressing mix" : 1   
  "french bread" : 1  
  "winter squash" : 1  
  "mexican seasoning" : 1  
  "mixed spice" : 1  
  "honey" : 1  
  "olive oil" : 1  
  "salt" : 1  
  "prepared pizza crust" : 1  
  "sausage patty" : 1  
  "eggs" : 1
```


Heuristic Search Approach

- ❖ Why Heuristic Search?
- ❖ How it works?
 1. Pre-filter Recipes
 2. Randomly generate meal plans
 3. Enforce basic constraints
 4. Evaluate each plan using scoring function
 5. Repeat for multiple iterations
 6. Keep the best scoring plan
- ❖ Heuristic Search is fast and scalable but trades off optimality and completeness compared to CSP.



Screenshots of Heuristic Approach

Smart AI Meal Planner (Heuristic)

Max Calories



Min Protein (g)



Select Diet

vegetarian

Generate Meal Plan 

Meal Plan

Breakfast: broccoli and cheese smashed potatoes

Lunch: lancashire roots and beans soup

Dinner: venison taco casserole

Nutrition

Calories: 1398.0

Protein: 157.0

Nutrition

Calories: 1398.0

Protein: 157.0

Plan Score

111.8

Grocery List

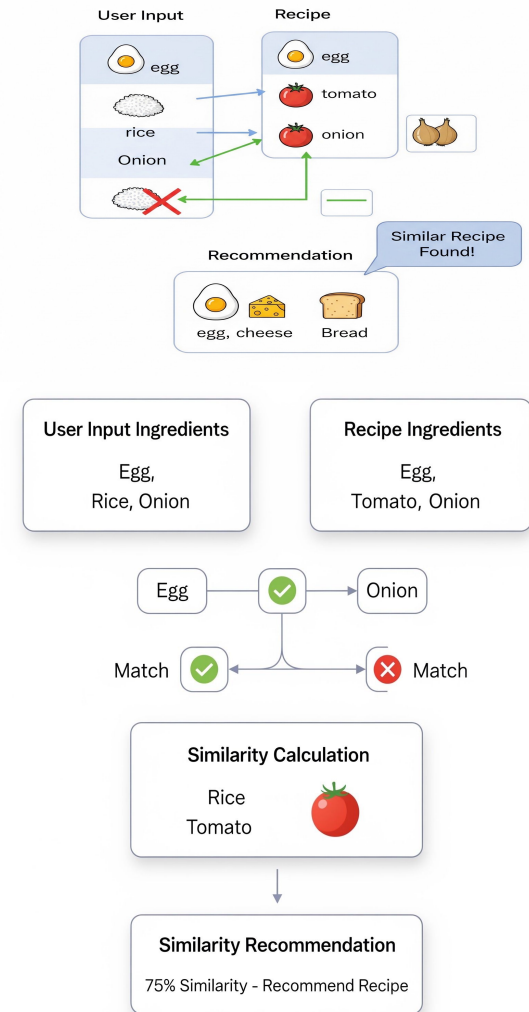
```
{
  "small potatoes" : 1
  "head broccoli" : 1
  "whole milk" : 1
  "butter" : 1
  "sharp cheddar cheese" : 1
  "salt & freshly ground black pepper" : 1
  "potatoes" : 1
  "carrots" : 1
  "onion" : 1
  "turnip" : 1
  "broad beans" : 1
  "kidney beans" : 1
  "baked beans in tomato sauce" : 1
  "bay leaf" : 1
  "thyme" : 1
}
```

Comparison between Heuristic and CSP

Feature	Heuristic Search	CSP (Backtracking)
Goal	Find best solution quickly	Find valid solution exactly
Speed	Fast	Slow
Optimality	Not guaranteed	Guaranteed (valid solution)
Completeness	No	Yes
Constraint Handling	Flexible	Strict
Scalability	High	Low
Determinism	No	Yes
Use Case	Real time apps	Small / critical systems

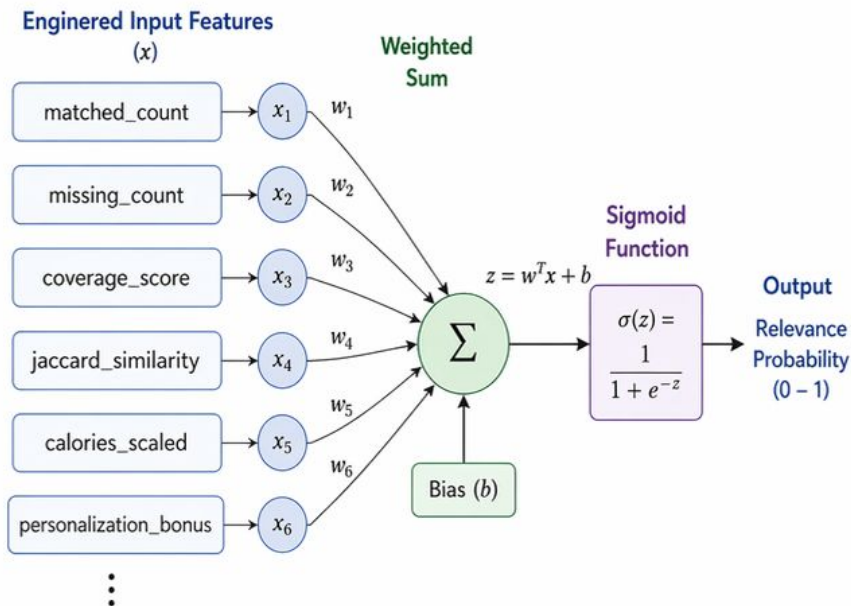
Content Based Recommendation

- Content-based recommendation suggests items based on similarity between item features and user input
- Uses ingredients as features
- Matches user ingredients with recipe ingredients
- Computes similarity using:
 - KNN score - uses nearest neighbors idea
 - Cosine similarity
 - Jaccard similarity
- Filters based on:
 - missing ingredients
 - calorie limits
- Outputs ranked recipes



Logistic Regression

Linear Model Overview



- The model learns weights ($w_1 \dots w_n$) and bias (b) to map features to a relevance probability.
- **Decision rule:** predict relevant if probability ≥ 0.5 .



Model Idea

- A linear classifier for binary relevance prediction.
- Combines features using a weighted sum.
- Applies sigmoid function to output probability (0–1).
- Interpretable coefficients indicate feature impact.



How It Works

- Computes $z = w^T x + b$ from input features.
- Passes z through the sigmoid function.
- Outputs probability of a recipe being relevant.
- Trained using cross-entropy loss and gradient descent.



Strengths

- Simple, fast, and easy to interpret.
- Feature weights show positive/negative impact.
- Stable generalization with less risk of overfitting.
- Strong baseline for tabular data.



Result (Test Set)

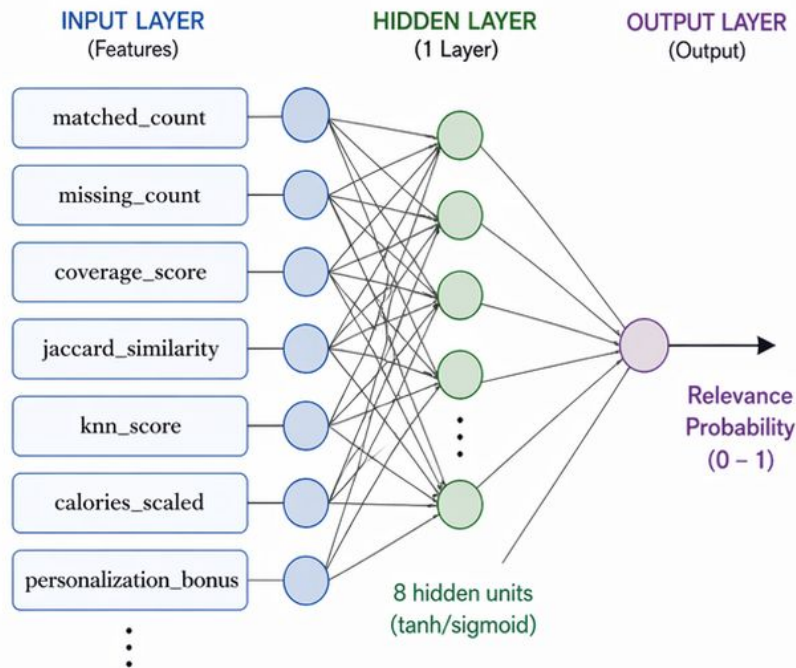
- Accuracy: 0.9685
- Precision: 0.9094
- Recall: 1.0000
- F1 Score: 0.9526



The model achieves perfect recall, meaning it finds all relevant recipes in the test set.

Test F1 = 0.9526

MLP Neural Network



- One hidden layer MLP
- Hidden units: 8
- Activation: Sigmoid
- Output: Probability (Sigmoid)
- Optimizer: Gradient Descent
- Learning rate: 0.035
- L2 penalty: 0.001
- Training epochs: 260



Architecture

- Feed-forward neural network with one hidden layer.
- 14 input features → 8 hidden units → 1 output.
- Hidden activation: Sigmoid
- Output activation: Sigmoid (predicts relevance probability)



How It Works

- Learns non-linear relationships and complex interactions between features.
- Each hidden unit captures a different combination of patterns from the inputs.
- Outputs a probability representing the likelihood that a recipe is relevant.



Strengths and Limitations

- **Strengths:** Can model complex feature interactions; flexible and expressive.
- **Limitations:** Less interpretable than Decision Tree or Logistic Regression; needs more tuning and data.
- **Observed Behavior:** Very high precision but lower recall, meaning it is conservative in predicting relevance.



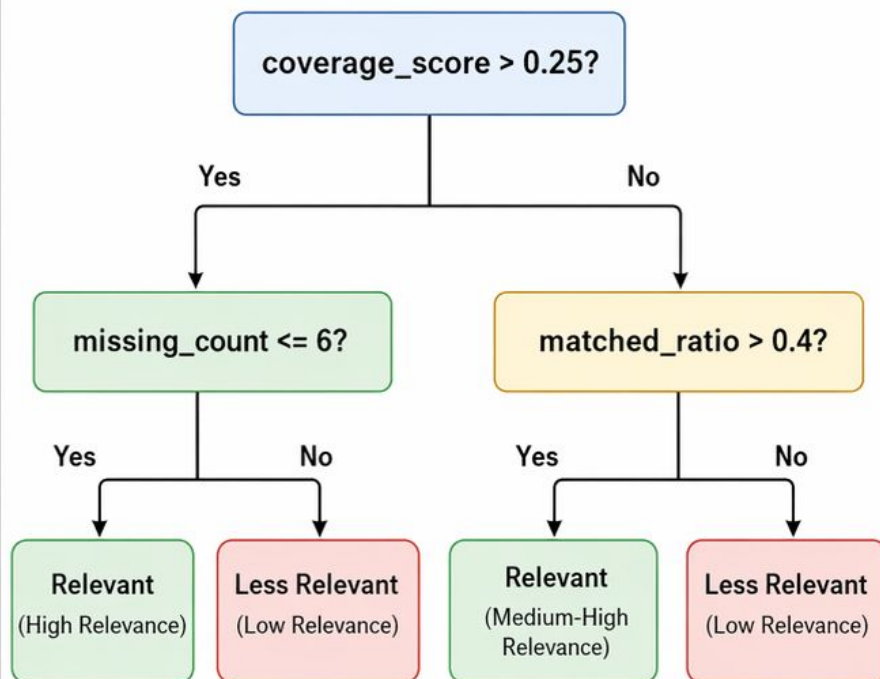
Result (Test Set)

- Accuracy: 0.8022
- Precision: 1.0000
- Recall: 0.6494
- F1 Score: 0.7874

The MLP achieves perfect precision but lower recall, indicating that it misses some relevant recipes.

Test F1 = 0.7874

Decision Tree



max_depth = 5, min_samples = 18



Input Features

- matched_count
- missing_count
- coverage_score
- jaccard_similarity
- knn_score
- calories_scaled
- personalization_bonus



How It Works

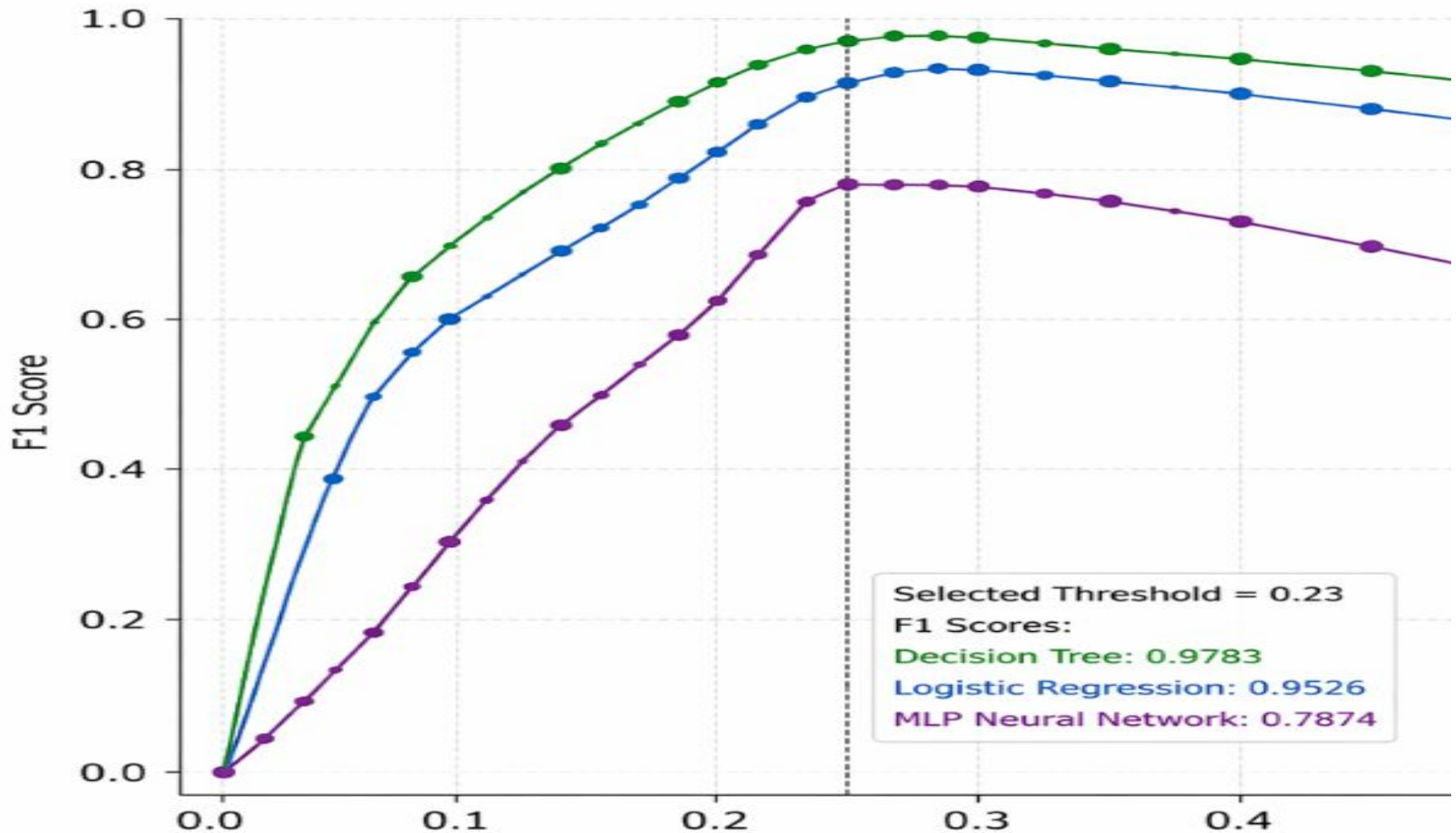
- Recursively splits data using feature thresholds.
- At each node, selects the split with the highest Gini impurity reduction.
- Captures non-linear relationships between features and recipe relevance.



Why Selected

- Models non-linear patterns more flexibly than Logistic Regression.
- More interpretable than neural networks (MLP).
- Achieves the best performance on held-out data.
- **Best Test F1 = 0.9783** (Accuracy = 0.9863)

Evaluations



Demo

Acknowledgements and References

- Dataset used -
<https://www.kaggle.com/datasets/shuyangli94/food-com-recipes-and-user-interactions>
- Research Papers referred -
<https://www.ijfmr.com/papers/2025/2/41432.pdf>
<https://www.ijraset.com/best-journal/smart-diet-planner>

Thank you!
Questions?